**REPORT ON**

**PROJECT BASED LEARNING**

**Carried out on**

# Vision-Based Gesture and Pedestrian Interactive Autonomous Car using AutoAuto Platform

**Submitted to**

**NMAM INSTITUTE OF TECHNOLOGY, NITTE**

**(Off-Campus Centre, Nitte Deemed to be University, Nitte – 574 110, Karnataka,India)**

**In partial fulfilment of the requirements for the award of the**

**Degree of Bachelor of Technology**

**In**

**Robotics & Artificial Intelligence**

**By**


| | |
|---|---|
| **Aashirvad A.Poojary** | **Bindu R.** |
| **NNM23RI010** | **NNM23RI013** |
| **Khushi K.** | **Vedant Mahalle** |
| **NNM23RI030** | **NNM23RI031** |

**Under the guidance of**

**DR. RASHMI P. SHETTY**

**Associate Professor**

**Department of Robotics & Artificial Intelligence**


NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

# CONTENT

## 1. Introduction

Autonomous systems are becoming increasingly important in modern transportation, particularly in ensuring safe and intelligent interaction between vehicles and humans. This project focuses on developing a vision-based interactive autonomous car that can respond to human gestures and pedestrian behavior in real time. The system uses a camera to continuously monitor the surroundings and applies image processing techniques to interpret human actions such as hand signals and pedestrian movement. Based on this interpretation, the car takes appropriate actions like stopping, turning, following, or reversing. The aim of this project is to demonstrate how basic computer vision techniques can be used to create an intelligent human-aware vehicle system without relying on complex deep learning models.

## 2. About AutoAuto Platform

The AutoAuto platform is a compact embedded system designed for developing and testing autonomous vehicle applications. It provides an integrated environment that includes a camera module, motion control capabilities, and a set of built-in APIs that allow easy interaction with hardware components. Functions such as car.forward(), car.left(), car.right(), and car.stop() enable direct control of the vehicle, while functions like car.capture() and car.detect_pedestrians() allow access to visual input and basic detection features. This platform is particularly useful for rapid prototyping of computer vision-based projects, as it eliminates the need for complex hardware configuration and allows developers to focus on logic and behavior.

## 3. Objective

The primary objective of this project is to design a system where a car can intelligently interact with humans using visual cues. The system is developed to recognize hand gestures such as a palm signal and respond by stopping or changing direction. Additionally, it is designed to detect pedestrians and exhibit dynamic behavior such as following a moving pedestrian and reversing when the pedestrian turns back. The goal is to simulate real-world interaction scenarios between humans and autonomous vehicles.

## 4. System Methodology

The system operates by continuously capturing frames from the camera using the AutoAuto API. Each frame is processed using OpenCV to detect motion and identify significant changes between consecutive frames. The process begins by converting the captured image into a numerical format suitable for processing. The current frame is compared with the previous frame using frame differencing, which highlights regions where movement has occurred. The resulting difference image is converted to grayscale and thresholded to isolate areas of significant motion.

Contours are extracted from the thresholded image, and their area is calculated. If a contour exceeds a predefined threshold, it is interpreted as a large object close to the camera, such as a human palm. This is used as a gesture signal. Based on this detection, the system decides whether to stop the car or change its direction.

For pedestrian interaction, the system uses built-in functions to detect the presence of a person. When a pedestrian is detected, the car follows the pedestrian. If the pedestrian turns back or changes direction, the system responds by reversing the car, thereby demonstrating interactive behavior.

## 5. Activities Performed

**Task 1: Understanding AutoAuto Platform and Basic Control**

The first stage of the project involved understanding the AutoAuto platform and its capabilities. The available APIs were explored using the list_caps() function to identify supported features such as camera access and vehicle control. Basic movement functions including car.forward(), car.left(), car.right(), and car.stop() were tested individually to ensure proper control of the vehicle.

In addition to movement, the camera functionality was verified using car.capture() to capture real-time frames. These frames were inspected to understand their format and how they could be processed using OpenCV. Initial trials were conducted to convert the captured frames into usable image formats using NumPy and OpenCV decoding techniques. This stage was important to establish a working connection between perception (camera input) and action (vehicle movement), forming the foundation for further development.

**Task 2: Gesture Detection using Palm Signal**

In the second stage, a gesture-based control system was developed using motion detection techniques. Instead of using complex machine learning models, a simpler and efficient approach was implemented using frame differencing. Consecutive frames captured from the camera were compared to identify changes in the scene. The difference image was converted to grayscale, blurred to reduce noise, and thresholded to highlight regions of significant motion.

Contours were then extracted from the processed image, and their area was calculated. A threshold value was defined such that any contour exceeding this area was interpreted as a large object close to the camera, typically representing a human palm. Based on this detection, control logic was implemented where the car either stopped or changed direction, such as turning right. Additional refinements were made to prevent continuous triggering of commands by introducing state variables, ensuring smooth and stable behavior. This task demonstrated how visual input can be used to control a system through simple gesture recognition.

**Task 3: Pedestrian Interaction and Behavioral Response**

The final stage focused on implementing interaction between the car and a pedestrian. The built-in pedestrian detection function provided by the AutoAuto platform was utilized to identify the presence of a person in the camera frame. Once a pedestrian was detected, the system was programmed to make the car follow the pedestrian by continuously moving forward.

To enhance interaction, additional logic was implemented to detect changes in pedestrian behavior. When the pedestrian turned back or changed orientation, the system interpreted this as a signal to stop or reverse. As a result, the car responded by moving backward, simulating a reactive and adaptive system. Multiple tests were conducted to fine-tune the responsiveness of the system and ensure that the transitions between forward and reverse movement were smooth. This task highlighted the ability of the system to not only detect human presence but also respond dynamically to human actions.

# 6. Code Implementation

## 6.1 Gesture-Based Control Code


**Fig 1.** Gesture-Based Control Code

## 6.2 Pedestrian Interaction Code


**Fig2.** Pedestrian Interaction Code

### 6.3 Explanation

The code uses OpenCV for image processing and the AutoAuto API for controlling the vehicle. Motion detection is implemented using frame differencing, and contour analysis is used to detect large objects such as a palm. Based on the detected input, appropriate commands are sent to the car to control its movement.

## 7. Results and Observations

The system successfully detects motion-based gestures and responds in real time. The palm detection mechanism works effectively when the hand is placed close to the camera, allowing the car to stop or change direction accordingly. The pedestrian interaction feature allows the car to follow a person and react dynamically to changes in movement. However, it was observed that the system is sensitive to lighting conditions and may detect large moving objects as gestures.

## 8. Limitations

The system relies on motion-based detection rather than advanced machine learning techniques, which limits its ability to accurately distinguish between different objects. It cannot specifically identify a human hand and may detect any large object as a gesture. Environmental factors such as lighting and background movement can affect performance. Additionally, the system is limited to basic interaction scenarios and does not support complex navigation.

## 9. Future Scope

The project can be enhanced by integrating advanced gesture recognition using machine learning models for improved accuracy. Additional features such as multi-gesture control, obstacle avoidance, and distance estimation can be implemented. The system can also be extended to support full autonomous navigation and more complex human-vehicle interactions.

## 10. Conclusion

This project demonstrates the implementation of a vision-based interactive autonomous car using the AutoAuto platform. By combining basic image processing techniques with real-time decision-making, the system successfully responds to human gestures and pedestrian behavior. The project highlights the potential of computer vision in developing intelligent and interactive autonomous systems.

## 11. Images Section

**Fig 3.** AutoAuto Car [Neelambari]

**Fig 4.** Pedestrains